

StealthSurvival

Game Design Document — TP Final de Programación con Motores de Videojuegos II

Título	StealthSurvival
Autor	Daniel Fimiani
Motor	Unreal Engine 5.6 (C++)
Género	Stealth / Infiltración top-down
Modo	Un jugador
Instancia	Examen Final de Programación con Motores de Videojuegos II

1. Descripción

StealthSurvival es un juego de infiltración en un único nivel cerrado, con cámara top-down. El jugador controla a un intruso que debe **robar un objetivo** (unos planos custodiados) y **escapar con ellos a la zona segura**, sin ser detectado por los guardias y las cámaras que vigilan el recinto. El acceso está bloqueado por puertas que exigen encontrar antes una **tarjeta de acceso** y un **código de escape**, por lo que la partida es también un recorrido de exploración con gating.

No hay combate frontal: la fuerza del jugador no está en pelear sino en **leer los patrones de vigilancia**, moverse en silencio, usar el entorno como cobertura y neutralizar amenazas por la espalda cuando no queda otra. Cada perceptor que te ve suma a una **barra de detección global**; si se llena, la partida se pierde.

Referencias de Game Design: la propuesta toma la tensión de sigilo de *Metal Gear Solid* (conos de visión, alerta progresiva, distracciones sonoras), el planteo top-down de infiltración de *Monaco* y *Commandos*, y la economía de riesgo de *Mark of the Ninja* (el jugador siempre sabe cuánto ruido hace y qué tan visible está).

Fantasía del jugador: ser el fantasma que entra, toma lo que vino a buscar y desaparece sin que nadie sepa que estuvo ahí.

2. Actores

Jugador

- Intruso** (`AStealthSurvivalCharacter`): personaje controlable. Se mueve en tres registros (caminar / correr / agacharse), cada uno con su velocidad y **radio de ruido**. Emite estímulos

de sonido y de vista que la IA percibe. Puede interactuar, hacer eliminaciones silenciosas, arrojar distracciones y volverse invisible (GAS).

Enemigos (IA autónoma) — al menos 3 en el nivel

- **Guardias** (`AStealthGuardCharacter` + `AStealthAIController`): enemigo principal, gobernado por un **Behavior Tree**. Patrulla por NavMesh, percibe por vista y oído, y escala su alerta (**Unaware** → **Suspicious** → **Alerted**). Investiga ruidos, persigue y abre puertas. Su **cono de visión** cambia de color (verde / amarillo / rojo). Se puede eliminar por la espalda. Sus parámetros se leen de un **Data Asset**, lo que permite crear tipos de guardia sin tocar código.
- **Cámaras de seguridad** (`AStealthSecurityCamera`): segundo perceptor, gobernado por un **State Tree**. Detección distinta: **solo visión**. Barre un arco fijo; al verte contribuye a la detección. Se puede desactivar desde un interruptor.

Actores del mundo

- **Objetivo** (`AStealthObjective`): los planos a robar; al tomarlos habilita el escape.
- **Zona de extracción** (`AStealthExtractionZone`): la zona segura / salida.
- **Tarjeta, código y puertas** (`AStealthKey` , `AStealthDoor`): gating de acceso.
- **Arbusto y escondite** (`AStealthCoverBush` , `AStealthHidingSpot`): cobertura y ocultación.
- **Interruptor de cámara** (`AStealthCameraSwitch`) y **objeto arrojadizo** (`AStealthThrowable`): desactivar cámaras y distraer guardias.

3. Mecánicas (implementadas en C++)

Más de siete mecánicas jugables, integradas entre sí y parte activa del núcleo:

- **Locomoción stealth contextual**. Caminar / correr / agacharse, cada modo con su velocidad y su **radio de ruido**. Agachado es silencioso; correr se escucha lejos.
- **Sistema de detección por IA**. Perceptores con **AI Perception**; cada uno que te ve cuenta como "watcher" y una **barra de detección global** sube/baja.
- **Eliminación silenciosa (takedown)**. Por detrás de un guardia (validado por producto punto y trace), neutralización con montage.
- **Distracción arrojadiza**. Un objeto que al impactar **emite sonido**; los guardias usan **EQS** para elegir un punto de inspección e ir a investigar.
- **Coberturas y escondites**. Arbustos (cobertura al agacharse) y muebles (ocultación total). La cámara aplica **fade de oclusión**.
- **Sistema llave → puerta**. Tarjeta de acceso y código de escape que abren puertas; gating del recorrido.
- **Interruptor de cámaras**. Desactiva cámaras enlazadas.
- **Invisibilidad (GAS)**. Habilidad activa con el **Gameplay Ability System** (`UGA_Invisibility`): por un tiempo la IA no te ve. Con cooldown en el HUD.

4. Metas y obstáculos

Meta principal: conseguir la tarjeta, robar los planos, encontrar el código y escapar a la zona de extracción con los planos.

Sub-metas / gating: tarjeta → puerta de los planos; código → puerta de salida. Desactivar cámaras, aprender rutas de patrulla.

Obstáculos: guardias con conos de visión y oído; cámaras de visión fija; puertas con llave; y la propia **barra de detección**. La **dificultad** elegida en el menú regula qué tan rápido sube esa barra.

Tensión central: velocidad vs. silencio. Correr llega antes pero hace ruido; agacharse es seguro pero lento.

5. Reglas

- El jugador **gana** si entra a la zona de extracción **teniendo los planos**.
- El jugador **pierde** si la **barra de detección se llena** (`DetectionLevel` = 1.0).
- La detección **sube** mientras al menos un perceptor te ve y **baja** cuando ninguno te ve. La tasa se multiplica por la **dificultad** (Fácil / Normal / Difícil).
- Un guardia solo se elimina **por la espalda** y en rango; queda fuera de juego.
- Las puertas con llave no se abren sin la llave/código correspondiente.
- El estado de la partida es explícito: **Playing** → **Won / Lost** (`EStealthMatchState`).
- El **tiempo de partida** se cronometra; al ganar se registra y se guarda el **mejor tiempo**.

6. Espacio (nivel y flujo de juego)

Un único nivel de gameplay (`Lvl_Stealth`): recinto cerrado con zonas de patrulla, mobiliario y arbustos que dan cobertura, los planos custodiados, puertas con llave y la zona de extracción como zona segura.

Flujo de juego completo:

- **Menú Principal** (`Lvl_MainMenu`, nivel separado) — Jugar / Salir y selección de **dificultad**; muestra el **mejor tiempo**. La transición se hace **en C++** (`UStealthMainMenuWidget::PlayGame` → `OpenLevel` → `Lvl_Stealth`).
- **Gameplay** — el ciclo de infiltración, con **HUD** activo (`UStealthHUDWidget` / `WBP_HUD`): barra de detección, objetivo actual (del Data Table), **timer**, ocultamiento y cooldown de invisibilidad.
- **Pantalla final** (`UStealthEndScreenWidget`) — al ganar o perder, pausa y muestra el resultado con el **tiempo de la partida** y el **récord**, con **Reintentar** o **Volver al menú**. Rejugabilidad sin reiniciar la aplicación.

7. Inteligencia Artificial (detalle técnico)

Herramienta	Uso en el proyecto
NavMesh + Pathfinding	Patrullaje y persecución (Runtime Generation: Dynamic).
Behavior Tree	Cerebro del guardia (patrulla → sospecha → alerta → persecución).
State Tree	Cerebro de la cámara (detección solo por visión).
AI Perception	Vista + oído en guardias; solo vista en cámaras.
EQS	Punto de inspección tras escuchar una distracción.
Smart Object	Puertas que la IA abre a su paso.
Teams (GenericTeamAgent)	Distingue jugador de IA para la percepción hostil.
ALSightTargetInterface	El jugador decide su visibilidad (cover, escondite, invisibilidad).

Los agentes son **autónomos desde el inicio**, reaccionan de forma **clara y progresiva**, y funcionan con **múltiples instancias activas** en simultáneo.

8. Arquitectura y buenas prácticas

Gameplay Framework

`AStealthSurvivalGameMode` (victoria/derrota, tick de detección, dificultad, timer) +
`AStealthSurvivalGameState` (detección, objetivo, llaves, match state, tiempo) +
`AStealthSurvivalPlayerController` (HUD, pantalla final, Enhanced Input).

Subsistemas (3, usando Game Instance Subsystem)

- `UStealthAlertSubsystem` (`UWorldSubsystem`) — patrón **Observer**: difunde alertas entre las IAs por delegate multicast.
- `UStealthItemRegistrySubsystem` (`UGameInstanceSubsystem`) — carga el **Data Table** de ítems y sirve su info (nombre, texto de HUD, ícono).
- `UStealthProgressSubsystem` (`UGameInstanceSubsystem`) — persiste la **dificultad** y el **mejor tiempo** con **SaveGame** a disco.

Data-driven (Data Assets / Data Tables)

- `DT_Items` (`FStealthItemData : FTableRowBase`) — info de ítems; el DataTable se declara en `UStealthGameSettings` (`UDeveloperSettings`).
- `UStealthGuardConfig` (`UPrimaryDataAsset`) — configuración de tipos de guardia.
- `UStealthGameSave` (`USaveGame`) — datos persistidos (dificultad, récord, stats).

GAS

Habilidad de **invisibilidad** (`UGA_Invisibility`) con cooldown vía Gameplay Effect, integrada al `UAbilitySystemComponent` del jugador y al HUD.

Asset Streaming

El `ItemRegistrySubsystem` carga el DataTable y **precarga los íconos por streaming asíncrono** (`FStreamableManager::RequestAsyncLoad` con `TSoftObjectPtr`), sin bloquear la carga del nivel.

Patrones de diseño (3+)

- **Observer** — `UStealthAlertSubsystem` (delegate multicast entre IAs).
- **State** — State Tree de la cámara + `EStealthAlertState` de los guardias.
- **Component / Composición** — `UInteractableComponent` reutilizable en objetivo, llaves y muebles.
- Complementan: **Interface/Strategy** (`IInteractable` / `IAISightTargetInterface`) y **Subsystem/Singleton**.

Comunicación entre clases C++

Interfaces (`IInteractable` , `IAISightTargetInterface` , `IGenericTeamAgentInterface`), delegados multicast (Observer) y referencias validadas con guardas de puntero.

Enhanced Input

Todas las acciones (mover, correr, agachar, takedown, arrojar, interactuar, invisibilidad) mapeadas con Enhanced Input.

Animación

`UStealthAnimInstance` con estados diferenciados (Idle / Walk / Run / Crouch).

9. Git y control de versiones

Proyecto versionado en Git (repositorio **StealthSurvival**), con **Git LFS** para los assets binarios de Unreal. El plugin RiderLink se mantiene fuera del repo (cada dev lo instala localmente). Assets acotados a lo estrictamente usado (estilo low-poly) para mantener el repo liviano dentro del presupuesto de LFS.

10. Mapeo a los criterios de evaluación

Criterio (mínimo)	Dónde se cumple
Ejecuta sin errores críticos	Flujo completo menú → gameplay → pantalla final probado.

≥ 7 mecánicas funcionales	8 mecánicas (sección 3).
Gameplay Framework	GameMode / GameState / PlayerController / HUD (sección 8).
Comunicación entre clases C++	Interfaces + delegados + referencias validadas.
Data Assets o Data Tables	<code>DT_Items</code> + <code>UStealthGuardConfig</code> + <code>UStealthGameSave</code> .
GAS	<code>UGA_Invisibility</code> con cooldown.
≥ 3 patrones de diseño	Observer, State, Component (+ Interface, Subsystem).
3 subsistemas + Game Instance Subsystem	Alert (World) + ItemRegistry + Progress (GameInstance).
Documentación detallada	Este documento.
Build del juego	Build empaquetada para Windows (entregada aparte).

Criterio (extra)	Dónde se cumple
Asset Streaming	Carga asíncrona de íconos vía <code>StreamableManager</code> + <code>TSoftObjectPtr</code> .
IA (NavMesh / State Tree / Perception)	Sección 7 (BT, ST, EQS, Smart Objects, Teams).
Validación de punteros / control de estados	Guardas de puntero en todo el código; <code>EStealthMatchState</code> .
Unreal Coding Standards / legibilidad	Nomenclatura UE, sin lógica muerta, headers acotados.

11. Diversión

El corazón divertido de StealthSurvival es la **tensión del casi-me-ven**: leer un cono de visión, calcular si me da el tiempo, tirar una distracción en el momento justo y colarme por el hueco que dejó el guardia. El jugador siempre tiene información clara (ruido, visibilidad, color del cono, barra de detección, timer) para decidir, y el juego premia la paciencia y la lectura del entorno por sobre los reflejos. La invisibilidad y el takedown dan la válvula de escape cuando el plan se rompe, manteniendo la sensación de estar siempre al borde pero nunca sin salida.